

BigLyp

The leap that defines you, Shapes your career

Task Manager — Edge-Native Full-Stack Project

Cloudflare
Workers

Next.js

Drizzle ORM

Cloudflare D1

TypeScript

Zod · Jose ·
bcryptjs

Duration	1 – 2 days
Difficulty	Medium
Deliverables	Live deployed URL (Cloudflare Pages + Workers) & public GitHub repository
Submission	Share both links over email/form. Repo must include a README with setup instructions and the live URL.

1. Overview

You are required to build a **Task Manager** — a full-stack web application that allows registered users to create, manage, and track their personal tasks. The backend must run as a **Cloudflare Worker** using a **Cloudflare D1** (SQLite) database, while the frontend is a **Next.js** application deployed to **Cloudflare Pages**. The project must be written in strict **TypeScript** throughout.

2. Required Tech Stack

Layer	Technology	Notes
Runtime	Cloudflare Workers	Edge serverless — no Node.js built-ins
Frontend	Next.js (App Router)	Deploy to Cloudflare Pages
Database	Cloudflare D1 (SQLite)	Must use D1 — no other DB
ORM	Drizzle ORM	drizzle-orm/sqlite-core dialect; migrations via drizzle-kit
Auth tokens	jose	Edge-compatible JWT sign/verify — do NOT use jsonwebtoken
Password hash	bcryptjs	
Validation	Zod	All inputs, API payloads, and env vars
Language	TypeScript (strict mode)	No any; no implicit types
HTTP client	Native fetch	Avoid axios — native fetch is available on the edge

3. Features to Build

3.1 Authentication

- User registration with name, email, and password.
- Password must be hashed with bcryptjs before storing.
- Login returns a signed JWT (via jose) with a 24-hour expiry.
- All task-related endpoints must be protected — validate the JWT on every request.
- Zod must validate all auth request payloads (e.g., reject missing fields, enforce email format).

3.2 Task Management (CRUD)

- A logged-in user can **create** a task with a title, optional description, status, and optional due date.
- A logged-in user can **list** their own tasks. Support optional query-param filtering by status.
- A logged-in user can **update** a task's title, description, status, or due date.
- A logged-in user can **delete** a task.
- Users must never be able to read or modify another user's tasks (enforce ownership checks).

3.3 Task Statuses

Each task must have one of three statuses: `todo`, `in-progress`, or `done`. This should be enforced at the DB schema level and validated with Zod on input.

3.4 Frontend Dashboard

- Register and Login pages with proper form validation.
- A task dashboard showing all the user's tasks in a clean, readable layout.
- Ability to create a new task from the dashboard (modal or dedicated page).
- Ability to change a task's status inline (e.g., a dropdown or button group).
- Ability to delete a task with a confirmation prompt.
- Filter tasks by status (All / Todo / In-Progress / Done) using tab/button UI.
- Display a friendly empty state when no tasks exist.
- Handle loading and error states gracefully — no silent failures.

4. Database Schema (Reference)

Below is the expected schema. You must implement this using Drizzle ORM's SQLite core. Migrations must be generated and applied via drizzle-kit.

users

Column	Type	Constraints
id	text	PRIMARY KEY — use a UUID / CUID
name	text	NOT NULL
email	text	NOT NULL, UNIQUE

password	text	NOT NULL — store bcrypt hash only
created_at	integer	Unix timestamp — default to now

tasks

Column	Type	Constraints
id	text	PRIMARY KEY — UUID / CUID
user_id	text	NOT NULL, REFERENCES users(id)
title	text	NOT NULL
description	text	Nullable
status	text	NOT NULL — one of: todo in-progress done
due_date	integer	Nullable Unix timestamp
created_at	integer	Unix timestamp — default to now
updated_at	integer	Unix timestamp — update on every PATCH

5. API Endpoints

All endpoints are served from the Cloudflare Worker. The Next.js

request/response bodies throughout.

frontend calls these directly. JSON

Auth Endpoints

Method	Endpoint	Auth?	Description
POST	/api/auth/register	—	Register a new user. Returns the created user (no password).
POST	/api/auth/login	—	Validate credentials, return a signed JWT.
GET	/api/auth/me	✓	Return the currently authenticated user's profile.

Task Endpoints

Method	Endpoint	Auth?	Description
GET	/api/tasks	✓	List all tasks for the authenticated user. Optional ?status= filter.
POST	/api/tasks	✓	Create a new task. Body: title, description?, status?, due_date?.
GET	/api/tasks/:id	✓	Get a single task by ID (must belong to the calling user).
PATCH	/api/tasks/:id	✓	Partially update a task (any subset of fields).

DELETE	/api/tasks/:id	✓	Delete a task (must belong to the calling user).
--------	----------------	---	--

6. Deployment Requirements

Backend — Cloudflare Worker

- Deploy the Worker using `wrangler deploy`.
- Bind the D1 database in `wrangler.toml` under `[[d1_databases]]`.
- Store the JWT secret as a `wrangler secret` — never hardcode it.
- Run `drizzle-kit migrate` against the production D1 database before final submission.

Frontend — Cloudflare Pages

- Connect the Next.js app to Cloudflare Pages via the dashboard or `wrangler pages deploy`.
- Set the Worker's deployed URL as an environment variable in Pages (e.g., `NEXT_PUBLIC_API_URL`).
- Ensure the production build passes — no TypeScript errors, no broken imports.

7. Submission Checklist

Checklist Item	Required	Details
GitHub Repository	✓	Public repo with clear commit history. Code must be yours — no copy-paste boilerplate without understanding.
README.md	✓	Must include: project overview, local setup steps, env variable list, migration command, and the live deployed URL.
Live Deployed URL	✓	Both the Next.js frontend (Cloudflare Pages) and the Worker API must be live and accessible.
D1 Database	✓	Production D1 migrations must be applied — the app must work end-to-end on the live URL.
TypeScript Strict Mode	✓	No TypeScript errors. Strict mode must be enabled in <code>tsconfig.json</code> .
No secrets in Git	✓	No API keys, JWT secrets, or credentials committed to the repository.
Zod validation	✓	All API inputs validated with Zod. Invalid requests return 400 with a clear error message.
Auth middleware	✓	All task endpoints return 401 when JWT is missing or invalid.
Ownership enforcement	✓	Users cannot access or mutate another user's tasks (must return 403 or 404).
Clean UI	✓	Dashboard is usable and reasonably styled. Loading/error states handled.

8. Evaluation Criteria

Area	Marks	What We Look For
Backend Architecture	25	Clean Worker structure, correct use of Drizzle ORM, D1 integration, and Zod validation on all routes.
Authentication	20	Correct JWT flow using jose, bcryptjs hashing, protected routes, and ownership enforcement.
Frontend Quality	20	Clean, functional Next.js dashboard. Proper API integration, loading/error states, and task filter UI.
TypeScript Quality	15	Strict mode, no any types, well-typed API responses and Drizzle schema.
Deployment	10	Both Worker and Pages live. D1 migrations applied. Env vars correctly set — no secrets in Git.
Code Quality & README	10	Readable code, logical structure, clear README with setup + live URL.
Total	100	

9. Tips & Common Pitfalls

Tips

- ✓ Start with the Worker API — get auth and one task endpoint working before touching the frontend.
- ✓ Use hono as your Worker router — it's lightweight, edge-native, and works great with D1.
- ✓ Generate a Drizzle schema first, run drizzle-kit generate, then drizzle-kit migrate.
- ✓ Use wrangler dev with --local to test D1 bindings locally before deploying.
- ✓ Store the JWT secret with wrangler secret put JWT_SECRET — read it from `env.JWT_SECRET` in the Worker.

Common Pitfalls to Avoid

Do NOT use jsonwebtoken	It doesn't work on the edge. Use jose — SignJWT and jwtVerify.
Do NOT use Node.js fs / path	You are in a Web Worker environment, not Node. Avoid any Node-specific built-ins.
Do NOT hardcode secrets	JWT_SECRET and any API keys must be injected via wrangler secrets or env vars — never in code.
Do NOT skip migrations	Run drizzle-kit migrate against production D1 before submission. The app must work live.
Do NOT ignore ownership checks	Every task read/write must verify the task's user_id matches the JWT sub claim.